



UNIVERSIDADE ESTADUAL DE MATO GROSSO DO SUL – UEMS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

CAIO EDUARDO GOUVEIA DIAS
RGM 47535

REDES DE COMPUTADORES
DOCUMENTAÇÃO DO TRABALHO

IMPLEMENTAÇÃO DE UMA APLICAÇÃO DE SERVIÇOS DE COTAÇÃO DE
PREÇOS DE COMBUSTÍVEIS

DOURADOS-MS
2025

1. Sumário

1. Sumario	[1]
2. Descrição de Algoritmos, Funções e Procedimentos	[2]
2.1. Servidor	[2]
2.1.1. Módulo Principal (<code>servidor_principal.py</code>)	[2]
2.1.2. Módulo Auxiliar (<code>servidor_auxiliar.py</code>)	[3]
2.2. Cliente (<code>cliente_principal.py</code>)	[3]
3. Decisões de Implementação Omissas na Especificação	[4]
4. Tratamento da Retransmissão de Mensagens	[5]
5. Testes e Análise de Resultados	[5]
5.1. Testes gerais, de Dados (Tipo 'D'), Pesquisa (Tipo 'P')	[5]
5.2. Analise	[6]
6. Telas de Captura e Exemplos de testes executados	[7]
7. Conclusão e Referências Bibliográficas	[9]
5.1. Conclusões e ideias de melhoria	[9]
5.2. Referências	[10]

2. Descrição de Algoritmos, Funções e procedimentos

O algoritmo se compõe de três arquivos principais, `cliente_principal.py`, `servidor_principal.py` e `servidor_auxiliar.py`. a modularização foi a solução para separar as responsabilidades e ter uma melhor visualização, principalmente por parte do servidor que se tornou mais extenso.

2.1. Servidor (`servidor_principal.py` e `servidor_auxiliar.py`)

2.1.1. Módulo principal (`servidor_principal.py`)

O servidor foi projetado em multithread e sua lógica de funcionamento baseada no padrão produtor-consumidor, assim garantindo que o processamento dos dados não interfira na recepção dos pacotes e no funcionamento do servidor.

Thread produtora (`funcao_servidor`): Sua função principal é escutar e esperar por pacotes UDP, assim que qualquer pacote chegar será chamada a função `verifica_pacotes` (a) para saber se o pacote chegou intacto do cliente ao servidor.

Caso o pacote seja válido, será retornado o próprio decodificado e será inserido junto com o endereço do cliente em uma fila (queue . Queue) segura para threads.

A partir disso serão resolvidos pela thread consumidora.

Thread consumidora (`funcao_processamento`): Da mesma maneira que o servidor aguarda por pacotes, o processamento aguarda por dados na fila. a implementação se torna intrinsecamente semelhante. a partir do momento que informações cheguem a fila, a função verifica o tipo de solicitação, caso seja Dados chamará a função `salvar_dados` (b). entretanto se for Pesquisa chamará a função `pesquisar_dados` (c) e ela retornará o posto com menor preço dentro do raio que por sua vez será enviado novamente ao cliente, mesmo que não tenha posto no raio, responderá ao cliente.

Principal("main") (`if __name__ == "__main__":`): Atribui os argumentos a porta e host, Administra a configuração do socket, o início e o fim das outras threads e lida com a interação do fechamento do servidor (o comando "exit").

2.1.2. Módulo auxiliar (`servidor_auxiliar.py`)

- a) `verifica_pacotes`: Calcula o checksum e envia um ACK ou NAK, assim decidindo se descarta o pacote e retorna (None) ou retorna o pacote preparado para o tratamento dos dados (montado como dicionário python)
- b) `salvar_dados`: responsável por fazer o armazenamento correto dos dados , assim como evitar conflitos e dados redundantes, caso o mesmo posto com mesmo combustível seja inserido, porém em preços diferentes, o preço será atualizado, por outro lado, caso o mesmo posto seja adicionado com mesmo valor e combustível será descartado. O armazenamento dos dados se consiste em um arquivo json, já que todo tratamento dos pacotes e dados são em dicionários que se adapta a json de uma forma natural, além da melhor visualização dos dados armazenados posteriormente.
- c) `pesquisar_dados`: Implementa a lógica de busca dado a um argumento que é a latitude, longitude e combustível. Carrega postos salvos no arquivo e os filtra por tipo de combustível, então calcula a distância em quilômetros usando a **Fórmula de Haversine**, quando a distância for menor que o raio de busca os dados essenciais são colocados em uma fila de candidatos. Ao final, utiliza a função `min()` com uma para encontrar e retornar o posto de menor preço dentro do raio de busca.

2.2. Cliente (`cliente_principal.py`)

`Principal("main") (if __name__ == "__main__"):` Aguarda pela entrada do cliente se for um "exit" encerra a execução, caso contrário chama `montar_pacote` (a) enviando como argumento a entrada do usuário para ser verificada e estruturada. depois que a entrada esteja preparada envia os arquivos com a função `envio_rdt` (b). Caso o tipo da mensagem seja Pesquisa, após o envio o algoritmo prepara o socket para receber a resposta do servidor, dado essa resposta decodifica o pacote e imprime o retorno.

- a) `montar_pacote`: Realiza o **parsing** da entrada do usuário (uma string separada por vírgulas), valida o formato e as informações e transforma em um dicionário Python estruturado.

- b) **envio_rdt**: decodifica a entrada para um pacote que de para enviar, e Implementa a lógica de transferência confiável (RDT) do lado do cliente, mais detalhada no **item 4**. em cada tentativa de envio do pacote ele simula uma aleatoriedade para testar o protocolo rdt enviando um checksum falso, já que seria muito difícil um pacote corromper para testes nas máquinas locais que usaremos

3. Decisões de Implementação Omissas na Especificação

Decisão de linguagem utilizada: A escolha da linguagem python foi devido a uma necessidade de aprendizagem vinda de projetos pessoais a serem desenvolvidos nos próximos meses. O desenvolvimento do trabalho se tornou uma maneira eficiente de lidar com essa necessidade, avançando em estudos com a linguagem python.

Estrutura do Pacote: Para garantir a integridade, foi adotada uma estrutura de pacote que contenha: um dicionário JSON externo com um campo **checksum** e os **dados** da mensagem, o campo **dados** contém outro dicionário (como uma string JSON) com as informações do posto {tipo, combustível, preço, latitude, longitude} respectivamente. O checksum é calculado sobre a string JSON dos dados internos apenas.

Protocolo de Confiança: Foi implementada uma versão do **RDT 2.0**, utilizando checksum para detecção de erros com base na somatória do pacote e feedback explícito (ACK/NAK) para confirmação, garantindo a entrega confiável sobre o UDP.

Formato de Dados: Optou-se pelo formato **JSON** para a serialização dos dados. Essa escolha foi feita por ser compatível com a escolha de armazenamento dos dados em variáveis que já estava fazendo uso de um dicionário python, como toda informação enviada seria um dicionário nada melhor do que a utilização de json para serialização dos dados que também torna-se mais seguro (diferente do **pickle**, não executa código arbitrário).

Retorno da verificação: Foi preferível fazer o retorno do ACKs e NAKs logo após o recebimento, para não empilhar dados que serão descartados no processamento e acelerar a resposta do cliente para não mantê-lo aguardando

implementação do RDT na pesquisa : realizei a implementação da verificação novamente quando o servidor retorna os dados de pesquisa para o cliente, porém acabaram com muitas funções redundantes que deveriam ser reimaginadas para ambos os lados. além da abertura de outro socket e outra porta. a fim de manter a estrutura inicial abandonei essa implementação.

4. Como foi tratada a Retransmissão de Mensagens

A retransmissão é gerenciada pelo cliente dentro da função `envio_rdt`, seguindo o princípio de parar o sistema até receber a confirmação que os dados chegaram:

1. O cliente envia um pacote.
2. Ele entra em um loop de espera, com um `timeout` de alguns segundos definido no soquete (`socket.settimeout()`) e fica aguardando feedback.
3. A retransmissão acontece em três circunstâncias:
 - **NAK**: Se o servidor responder explicitamente que o pacote chegou corrompido, ou seja, o resultado do checksum foi incoerente com o cálculo realizado no pacote no servidor.
 - **Resposta Inesperada**: Se a resposta não for um “ACK” ou “NAK”, assume-se que o feedback deve ter sido corrompido.
 - **socket.timeout**: Se o servidor não responder a tempo, porém nessa circunstância possivelmente o servidor estaria offline.
4. Um contador de tentativas limita a retransmissão a 3 vezes para evitar loops infinitos caso o servidor esteja offline.
5. o cálculo do checksum para confirmar se baseia em uma variável chamada checksum que calcula o valor total da mensagem, porém envia apenas o último byte, ou seja 8 bits. já que se torna mais difícil a perda de um único byte durante o transporte

5. Testes e Análise

5.1. Testes gerais, de Dados (Tipo 'D'), Pesquisa (Tipo 'P')

- **Testes globais:**
 - Confirmou-se que entradas em formato inválido foram rejeitadas pelo cliente antes do envio.
 - Testou-se que dados inválidos fossem rejeitados pelo cliente antes do envio.
 - Confirmou-se que o ip e porta não travassem o código caso incorretas

- **Testes de Dados (Tipo 'D'):**
 - Verificou-se a inserção correta de dados no arquivo `dados.json` em estado vazio.
 - Validou-se a atualização de preço de um posto existente, e confirmando que nenhum posto foi duplicado, apenas atualizado.
 - Testou-se o envio de dados idênticos ao servidor, onde o sistema identificou e não realizou a adição desses dados.
- **Testes de Pesquisa (Tipo 'P'):**
 - Realizou-se uma busca que se encaixou em múltiplos postos dentro do raio, e a função identificou e retornou o de menor preço.
 - Testou-se uma busca onde o posto mais barato estava fora do raio, e o sistema retornou um resultado mais caro, mas que ainda era o mais barato dentro do raio.
 - Validou-se o retorno nulo (`None`) para pesquisas onde nenhum posto se encaixava na pesquisa (tipo de combustível ou distância).

5.2. Analise

Muitos testes foram feitos com trajetos retos de um ponto ao outro, pois assim teria a confirmação que a distância retornada era precisa ao analisar a distância calculada do google maps dado a duas coordenadas de extremidades em uma rua com trajeto contínuo.

A análise dos testes demonstrou que:

- Nenhuma informação inválida será passada ao servidor a fim de otimizar o lado do processamento e comunicação.
- A lógica de "adicionar ou atualizar" está funcionando como esperado.
- A busca geolocalizada com a Fórmula de Haversine é precisa e retorna corretamente todos os postos dentro do raio.
- A distância calculada pela fórmula é preciso se comparado com coordenadas de dois pontos retos no google maps
- Os dados são armazenados, retirados ou modificados sem perder a integridade.

6. Telas de captura e exemplos de testes executados

Para os testes foram usadas coordenadas reais de postos em dourados, a maioria em uma única rua (marcelino) para receber a distância mais precisa sem a interferência dos trajetos. algumas entradas usadas nos testes abaixo:

Dados

D,0,5.20,-22.227087, -54.80274
 D,0,5.30,-22.227267, -54.804442
 D,0,4.90,-22.9, -55.9
 D,0,4.60,-22.213998, -54.813512
 D,0,4.70,-22.224012, -54.777096

Pesquisa

P,0,10, -22.231901, -54.821499
 P,0,5,-22.231980,-55.822712
 P,0,1.1,-22.227395, -54.805495

Envio D,0,4.70,-22.224012, -54.777096:

```
>> Enviando Dados:
{'tipo': 'D', 'combustivel': 0, 'preco': 4700, 'latitude': -22.224012, 'longitude': -54.777096}

simulacao ACK/NAK corrompido!
❌ NAK recebido. Reenviando a mensagem...

✅ ACK recebido. Mensagem entregue com sucesso!
```

Na resposta ocorre uma corrupção do pacote, logo após receber um NAK o pacote é enviado novamente e o novo posto é armazenado.

```
❌ Pacote recebido corrompido. Enviando NAK.

✅ Pacote recebido. Enviando ACK...

>> Armazenando:
{'combustivel': 0, 'preco': 4700, 'latitude': -22.224012, 'longitude': -54.777096}
Novo posto adicionado.
```

```
{
  "combustivel": 0,
  "preco": 4700,
  "latitude": -22.224012,
  "longitude": -54.777096
}
```

Enviando o mesmo local, porém houve um aumento no preço:

```
>> Enviando Dados:
{'tipo': 'D', 'combustivel': 0, 'preco': 4800, 'latitude': -22.224012, 'longitude': -54.777096}

✅ ACK recebido. Mensagem entregue com sucesso!
```

Dado a mesma coordenada e preço o mesmo foi atualizado:

```
✅ Pacote recebido. Enviando ACK...

>> Armazenando:
{'combustivel': 0, 'preco': 4800, 'latitude': -22.224012, 'longitude': -54.777096}
Mesmas coordenadas, preço atualizado: 4700 -> 4800
```

```
{
  "combustivel": 0,
  "preco": 4800,
  "latitude": -22.224012,
  "longitude": -54.777096
}
```

Envio de uma coordenada próxima a um posto com baixo raio de busca
P,0,0.2,-22.227395, -54.805495:

```
>> Enviando Pesquisa:
{'tipo': 'P', 'combustivel': 0, 'raio_busca': 0.2, 'latitude': -22.227395, 'longitude': -54.805495}

✅ ACK recebido. Mensagem entregue com sucesso!

>> Resposta da pesquisa recebida do servidor:
Preço: 5300 reais, Distância: 0.109 km a {'latitude': -22.227267, 'longitude': -54.804442}
```

O retorno constitui em um posto mais caro, porem ainda o menor preço dentro do raio de busca.

Realizando uma pesquisa no meio das coordenadas, e com um raio que abrange todos os postos cadastrados P,0,10, -22.231901, -54.821499

```
>> Enviando Pesquisa:
{'tipo': 'P', 'combustivel': 0, 'raio_busca': 10.0, 'latitude': -22.231901, 'longitude': -54.821499}

✅ ACK recebido. Mensagem entregue com sucesso!

>> Resposta da pesquisa recebida do servidor:
Preço: 4600 reais, Distância: 2.154 km a {'latitude': -22.213998, 'longitude': -54.813512}
```

O retorno será o menor preço deste combustível na cidade, nos postos que estão cadastrados.

7. Conclusão e Referências Bibliográficas

7.1. conclusões e ideias de melhoria

O projeto demonstrou um sistema cliente-servidor funcional sobre o protocolo UDP, mesmo que a transferência padrão UDP possa não garantir a integridade dos pacotes, através da implementação de um protocolo de transferência confiável (RDT 2.0) foi possível preencher as lacunas e necessidade de integridade deste protocolo.

Mesmo que não tenha sido resolvido o problema de que o feedback a respeito do pacote possa ser corrompido nas mesmas circunstâncias durante seu retorno, o que em um caso de “ACK’s” corrompidos geraria dados duplicados pois o cliente enviará novamente os dados até receber uma confirmação plausível dos mesmos.

A arquitetura Produtor-Consumidor no servidor mostrou-se adequada para o funcionamento correto e contínuo do servidor, separando o recebimento do pacote e seu processamento, garantindo que a aplicação mantenha-se responsiva.

Ideias para Melhorias e Trabalhos Futuros:

Olhando para frente, o projeto atual serve como uma boa base para diversas expansões para aprofundar nesse estudo como:

- **Melhorias na estrutura:** As escolhas de implementação como o uso de JSON e a lógica de dicionários aplicada facilitaram no desenvolvimento, entretanto senti que talvez o pacote seja pouco intuitivo para montagem e em uma aplicação em uma maior escala, podendo resultar em problemas de manutenção.
- **Preencher lacunas do RDT 2.0:** Implementar o RDT 3.0, adicionando timers e números de sequência(1,0) no lado do cliente para lidar também com a perda do feedback do pacote (ACKs corrompidos).
- **Criptografia de dados:** Adicionar uma camada simples de criptografia (usando bibliotecas como `cryptography`) ou até mesmo uma compactação de huffman para dados mais extensos no propósito de aprender a transferir esses dados de maneira mais segura.
- **Gerenciamento de Múltiplos Clientes:** Desenvolver um sistema capaz de gerenciar múltiplos clientes com diferentes processos ou threads e não limitando a uma fila de execução.

O sistema final é capaz de enviar, receber, validar, armazenar e consultar dados de forma consistente através de uma transferência UDP, atendendo aos requisitos e especificações propostas por esse projeto. Em minhas considerações finais, o trabalho não apenas atendeu aos requisitos especificados, mas também abriu possibilidades para aprofundar os estudos em áreas de redes de computadores.

7.2. Referências:

- Exemplos de implementação: [Redes de Computadores Rubens](#)
- Aplicações do RDT: [transporte.pdf](#)
- Makefile: [GNU make](#)
- Guia mais visual das funções socket: [Socket Programming in Python \(Guide\)](#)
- Documentação Oficial do Python 3. (2024), Módulos:
 - socket: [socket — Low-level networking interface — Python 3.13.5](#)
 - exceptions: [Built-in Exceptions — Python 3.13.5](#)
 - threading: [threading — Thread-based parallelism — Python 3.13.5](#)
 - queue: [queue — A synchronized queue class — Python 3.13.5](#)
 - json: [json — JSON encoder and decoder — Python 3.13.5](#)
- E por fim, é claro para algumas identificações de erros, encaminhamento para sites úteis e pesquisas ágeis : [ChatGPT](#)
"Em tempos de engano universal, dizer a verdade é um ato revolucionário." -
George Orwell